METROINSIGHTS

Complete Source-Code Deep Dive

Phase 2 study material | Repository verified 14 October 2026

Goal: be able to trace a user selection from the dashboard through Flask, forecasting, scheduling and network travel-time code, and explain what each result does and does not mean.

How to use this guide

Read the repository map first, then follow the request flow in order. For interview preparation, practice explaining the critical functions aloud before reviewing the questions and exercises. File and function names are emphasized so you can locate each behavior in the checkout.

Accuracy boundary. MetroInsights is a prototype built around generated footfall scenarios and hand-authored operating assumptions. The model predicts the generated target, and the scheduling logic is a deterministic scenario calculator. It is not connected to observed ridership or live DMRC operations.

Repository: `dmrc_footfall_predictor` | Flask + scikit-learn | Branch `main` | HEAD `4405d16`

1. Repository map and execution entry points

The app is organized around a small Flask runtime and a separate offline training pipeline. Generated training data is not needed for normal prediction after model artifacts have been created.

Path	Role / interview-relevant detail
run.py; Procfile	Local and hosted entrypoints. run.py imports the Flask app; Procfile uses `gunicorn run:app`.
app/__init__.py	Creates the Flask app and imports routes so decorators register endpoints.
app/routes.py	HTTP boundary: parse controls, call forecast/schedule/calendar/graph code, return HTML or JSON.
app/forecasting.py	Loads saved model, metadata and station directory at import; constructs feature rows and predictions.
app/scheduling_optimizer.py	Turns line-level demand plus CSV assumptions into headway, frequency and capacity scenario outputs.
app/calendar_context.py; operating_hours.py	Date labels/occasion flags, and 05:00-23:30 operating-window gate.
app/network_topology.py; travel_graph.py	Ordered line stops/transfers; graph build and multi-source Dijkstra.
app/templates/; app/static/	Jinja pages, CSS, GIF, browser JavaScript, Chart.js interface.
pipeline/01...; 02...; 03...	Clean inputs, generate synthetic labeled examples, then train/evaluate and save model artifacts.
data/raw, data/cleaned, data/synthetic, models/	Small source/derived inputs, generated line-share metadata, saved inference artifacts and evaluation metrics.
read.md; DATA_AUDIT.md; requirements.txt	Setup/deployment notes, provenance and limitations, dependency declaration.

Startup sequence

```
Gunicorn or python run.py
  -> run.py imports app
  -> app/__init__.py creates Flask instance
  -> imports app.routes (registers routes)
  -> routes loads station metadata and builds travel graph
  -> forecasting loads joblib model + metadata + station CSV
  -> request handlers become available
```

Important consequence: several files are read and the graph/model are initialized during process startup/import. A missing required artifact can prevent the service from starting; it is not necessarily an error that appears only after a prediction request.

2. Component dependencies and data contracts

Component	Input	Output / downstream use
Forecasting	station + datetime/hour; saved feature schema/model	integer demand/profile; route sends demand to optimizer and browser
Line allocation	station + station line list; JSON share metadata	line-specific demand used by schedule calculation and breakdown chart
Optimizer	line, station, hour, weekday/workday, demand, ops row	headway, tph, capacity/utilization, wait, staffing/fleet scenario, narrative
Travel graph	origin station; topology and coordinates	minimum estimated minutes per reachable station; frontend searchable list
Calendar context	Python date	weekday/workday/holiday/occasion context shown by UI and passed into optimizer

Two runtime data facts

• Ordinary inference uses committed model/metadata and small CSV/JSON inputs. It does not load the large generated training CSV.

• Retraining does require the synthetic CSV; pipeline 02 can regenerate it. The ignored local CSV is therefore a training asset, not an app-serving database.

• There is no database or user-scenario persistence. A request computes a response and returns it to the browser.

No test suite, CI workflow, ORM/database, Dockerfile, or dependency lockfile is tracked in this repository snapshot. The declared scikit-learn range and joblib artifact compatibility should be managed carefully for repeatable deployment.

3. HTTP request flow: from controls to response

The dashboard is a thin client. Its main job is collecting state, requesting a scenario, and mapping returned fields to the page. Flask coordinates domain modules and returns a single JSON object.

Frontend sequence in `app/static/js/dashboard.js`

```
DOMContentLoaded -> initControls() -> fetchPrediction()
control change -> fetchPrediction()
fetchPrediction(): POST /api/predict
  request sequence id prevents stale response overwrites
  response JSON -> currentData -> renderDashboard()
renderDashboard() -> summary + chart + line breakdown + travel table
```

Backend sequence in `app/routes.py: api_predict()`

```
1. Read JSON body (or query parameters on GET).
2. Normalize station and parse date/time; parse failures return HTTP 400.
3. Get calendar context; check selected minute against service hours.
4. Open: infer selected demand and full hourly profile. Closed: selected demand is zero.
5. Apply optional line focus; split interchange demand when appropriate.
6. Run schedule scenario(s).
7. Query Dijkstra travel-time estimates from the selected station.
8. Return JSON fields for date context, demand, service, profile, travel, provenance.
```

Race protection matters: browser requests can overlap when someone changes station/time quickly. A monotonically increasing request id lets the newest response win and prevents a slower old response from repainting the page.

The backend parses a valid time even if it is outside service hours; the operating-hours rule handles that as a closed scenario. This is the right place for the overnight-zero behavior, rather than silently clamping the user’s requested time.

4. Backend walkthrough: routes and service hours

`app/routes.py` owns request handling, not the underlying algorithms. It imports modules and translates their results into HTTP responses.

Function	What it does	Failure / edge behavior
_parse_datetime(date_str,time_str)	Defaults date to today in Asia/Kolkata and time to 09:00; strict date format; parses hour/minute.	Malformed date/time raises and API maps it to 400; does not itself clamp time to operation window.
_hour_band(profile,hour,minute)	Labels a point against the station's predicted daily profile and closure state.	Assumes hourly profile indexes align from 05:00; edge buckets need care around 23:00.
landing() / index()	Render landing and dashboard pages; dashboard provides station metadata/context.	Template/render or startup data errors are server-side failures.
get_footfall()	Legacy form POST path that renders a synchronous result page.	Distinct from modern asynchronous dashboard API.
api_predict()	Main orchestration endpoint; supports GET/POST, returns JSON forecast/service/travel data.	Bad parameters -> 400; unknown station behavior depends on downstream fallback/data; broad errors should be surfaced in logs.
api_stations()	Returns station directory data for controls.	Depends on station CSV loading.
api_model_metrics()	Returns saved comparison metrics with provenance flags.	Metrics are stored artifact results, not recomputed live.

Operating-hours rule

`app/operating_hours.py:is_service_operating(hour, minute)` is the shared gate. Its modeled window is 05:00 through 23:30: 23:30 is open; 23:31 through 04:59 is closed; 05:00 reopens. It is a project-wide simplification, not a line-specific published timetable.

Closed-time request path

The selected-time forecast becomes zero and schedule outputs are overridden as closed/no service. A full-day chart can still show operating-hour estimates, because it represents the day's profile rather than the selected 02:02 instant. Avoid confusing the profile with rides occurring at the selected time.

5. Data preparation and synthetic-data generation

The pipelines are explicit scripts, run in numbered order when rebuilding data/model artifacts. They are not part of web request execution.

Script / critical functions	Walkthrough
pipeline/01_audit_and_clean_data.py clean_stations()	Reads raw station rows, normalizes names/coordinates and station metadata, incorporates authored interchange definitions, then writes cleaned station/typology-compatible outputs.
create_operations_dataset()	Writes assumed per-line operating parameters (capacity, signaling/policy bounds, baseline headway, fleet and turnaround). Treat those as scenario inputs until externally sourced.
pipeline/02_generate_calibrated_dataset.py build_line_distribution_model()	Creates line-share weights using line throughput weights and route length, normalizes shares, and fixes rounding residuals. Shares are constructed assumptions, not observed transfer counts.
compute_station_weights()	Multiplies line weights by interchange and named-hub factors to allocate system-wide demand across stations.
generate_annual_footfall_dataset(seed=42)	Uses a seeded NumPy generator, calendar/day/month/event factors, station typology and hourly profiles, then Poisson samples each expected hourly count. Writes synthetic rows and line-share/typology outputs.

Why there are millions of rows

The generator spans 728 dates x 228 stations x 19 modeled hours (05:00-23:00), yielding 3,153,696 station-hour records. Each row is an output of a programmed demand recipe; the count is synthetic even where station names and coordinates are grounded in the station seed file.

Useful interview explanation: “I separated data engineering from serving. The numbered pipeline prepares metadata, generates reproducible scenario labels with a fixed seed, then trains artifacts. The deployed app loads the saved estimator; it does not regenerate millions of examples per request.”

Main synthetic-data edge cases

• A fixed random seed improves reproducibility, but does not make the simulation realistic by itself.

• Monthly/event/day multipliers may interact; inspect whether each factor is applied before or after Poisson sampling when changing the generator.

• Legacy duplicate CSV output can create confusion about which generated file is canonical.

• A line-share residual correction can slightly alter one line’s share to force the total to exactly 1.

6. Model preparation, training and evaluation

`pipeline/03_train_and_evaluate_forecaster.py` converts generated rows into the estimator's saved input schema.

prepare_annual_data()

• Loads synthetic target rows plus cleaned station information, typologies, and line-share context.

• Builds date parts and cyclical encodings for hour, weekday and month; represents station, line and typology with model-ready indices.

• Creates historical means at station, station-hour-weekday and station-month levels. Training features use earlier history; the held-out period uses aggregates derived from training dates to reduce leakage.

• Returns ordered feature columns and target values so training and inference can agree on feature shape.

train_and_evaluate_model()

Splits data chronologically at 2026-10-01, compares simple station and historical baselines plus Ridge, then fits `HistGradientBoostingRegressor` with 250 iterations, learning rate 0.06, max depth 9, minimum leaf 25, L2 0.05 and seed 42. It saves the model, feature metadata, and metrics JSON.

compute_metrics(y_true,y_pred)

Computes MAE, RMSE, R^2 and MAPE. MAPE excludes zero targets to avoid division by zero. Each metric describes agreement with generated holdout labels, because both train and holdout samples come from the same synthetic generation process.

Interview question	Accurate answer shape
Why a chronological split?	It tests a later date range against earlier examples and better approximates future prediction than randomly mixing dates.
Does $R^2 = 0.9947$ prove real accuracy?	No. It measures fit to synthetic holdout labels generated from the same rules; observed ridership validation is absent.
Why persist metadata?	Inference needs matching station/line/category indices and exact feature ordering, not only estimator weights.
What could leak?	Historical aggregates, encoders or preprocessing fitted using holdout/future targets; audit that every aggregate is constructed from allowed prior/training rows.

Model artifact compatibility is a deployment concern: a joblib estimator is Python/scikit-learn-version sensitive. Keep training and serving environments aligned and treat joblib files as trusted executable serialization.

7. Runtime forecasting: feature row to plotted curve

`app/forecasting.py` imports the committed estimator and metadata once, then exposes a small inference interface.

Function	Detailed behavior
get_station_info(station)	Returns known station metadata. Unknown names take a fallback path with line “Unknown”, no represented lines and null coordinates; this avoids a lookup crash but may not be a valid forecast identity.
predict_hourly_footfall(station, dt_obj, hour)	Returns 0 outside hour bounds; derives date/time encodings, station/line/typology indices and historical means; fills columns in saved metadata order; calls model.predict(); clamps negative output to 0 and returns an integer.
predict_footfall_with_minute_interpolation(...)	For open-time calculations, linearly blends hourly estimates at hour and next hour using minute/60. Next-hour lookup is capped at 23. The route’s service gate handles closed selected times.
predict_station_full_day_profile(station,dt_obj)	Predicts 19 points, hours 5 through 23; includes baseline/history values, daily aggregate/peak context and line curves derived from shares. Used by chart and schedule contexts.
get_model_evaluation_summary()	Reads saved model metrics summary for provenance/reporting surfaces; does not evaluate the model per request.

Example walkthrough: selected time 08:30

```
p08 = predict_hourly_footfall(station, date, 8)
p09 = predict_hourly_footfall(station, date, 9)
demand_08_30 = round(p08 + (p09 - p08) * 0.5)
# feature column order comes from saved metadata
```

The interpolation gives a smooth within-hour estimate, not a measured half-hour count. It can produce nonzero historical/profile values around closure if called directly without the route gate; callers must respect the shared service-hours rule.

Critical shape contract: training and runtime must agree on feature names, order, categorical index maps and expected dtypes. If metadata and estimator artifacts drift apart, prediction may fail or silently use wrong features.

8. Scheduling optimizer: rules, formulas, limitations

`app/scheduling_optimizer.py` translates forecast demand into a deterministic service scenario. It is not an ML model, dispatch solver or timetable simulator.

Core steps in optimize_line_schedule()

```
fleet_cap_tph = floor(fleet_size * 60 / turnaround_min)
signal_cap_tph = floor(60 / min_signal_headway_min)
max_tph = min(fleet_cap_tph, signal_cap_tph)
min_tph = ceil(60 / max_policy_headway_min)

through_load = 0.65 if peak else 0.40
boarding_clearance = max(int(comfort_capacity * (1-through_load)), 250)
required_tph = ceil(line_demand / boarding_clearance)
# compare with baseline, clamp limits, round to practical headway
utilization = line_demand / estimated_clearance_capacity
```

Key helpers

Function	Purpose / boundary
is_peak_hour(hour, day_of_week, is_workday)	Workday integer-hour peak windows 08-10 and 17-20. Hour-level labeling means the whole 20:00 hour is peak, although comments describe a peak ending at 20:30.
get_interchange_line_split(station, all_lines)	Loads saved synthetic split where available; falls back to equal allocation. A split is not measured origin-destination boarding.
round_to_practical_headway(value,min,max)	Clamps/rounds to a practical headway set while respecting configured bounds. Rounding can change the raw demand-derived frequency; outputs should be recomputed after rounding.
optimize_station_schedule(...)	Allocates demand to lines, runs per-line scenarios and selects a primary summary. Closed service overwrites operational recommendation outputs.

Interpretation boundary

Utilization is projected line demand divided by modeled boarding-clearance capacity. It does **not** track how many riders are already onboard when the train reaches a station, nor exits at that station. There is no train-by-train load state, OD matrix, train location or live timetable input. “Comfortable capacity” is a scenario parameter.

Edge cases: invalid or missing operations rows may use fallback matches; a station with no lines can leave no per-line results and make primary selection fail; extremely low demands may still be limited by minimum service rules; closed-time handling should be applied consistently by the caller and in optimizer output.

9. Network topology and shortest-time estimates

`app/network_topology.py` stores ordered stop lists and hand-authored interchanges. `app/travel_graph.py` maps those labels to station records and builds a graph whose nodes are station-line pairs.

Graph construction

```
for each line stop sequence:
  connect each adjacent (station, line) pair in both directions
  edge minutes from coordinate distance, or fallback
for each interchange:
  connect that station's line-nodes with transfer penalty
```

Distance estimate

For usable coordinates, `_haversine_km()` estimates great-circle distance. Segment time is roughly distance / 32 km/h x 60, floored to at least one minute. Segments above eight minutes and missing/repeated placeholder coordinates use a 2.5-minute fallback. Inter-line interchange edges use a fixed five-minute transfer penalty.

Dijkstra query

`shortest_times_from(graph, station_name)` starts from all line-specific nodes representing the origin. A min-heap processes lowest accumulated cost first; each reachable neighbor is relaxed when a shorter distance is found. The function then collapses line-specific nodes to the minimum time per destination station and removes the origin.

```
dist = {origin_line_node: 0 for origin_line_node in origin_nodes}
heap = [(0, node) for node in origin_nodes]
while heap:
    minutes, node = heappop(heap)
    if minutes != dist[node]: continue
    for neighbor, edge_minutes in graph[node]:
        candidate = minutes + edge_minutes
        if candidate < dist.get(neighbor, infinity):
            dist[neighbor] = candidate
            heappush(heap, (candidate, neighbor))
```

What the result does not include

- No itinerary/path reconstruction: the graph returns times, not a station-by-station route.
- No schedule waits, direction-specific running times, transfers by platform, disruptions, or time-of-day service.
- Topology completeness and transfer aliases are authored project data and need validation.
- Coordinate distance is a geometric proxy; it is not the measured rail distance.

10. Browser rendering and API contract

`app/static/js/dashboard.js` initializes controls, requests the current scenario and updates existing page elements. Jinja templates provide the initial shell and station metadata; Chart.js draws visualizations.

Frontend function/group	Purpose
initControls()	Binds station/date/time/slider/line/preset/search listeners and triggers requests when relevant values change. The slider selects whole hours; the clock input can select minutes.
updateLineSelectOptions()	Rebuilds the line selector from selected station metadata and resets it to “all lines” context.
fetchPrediction()	Builds request values, displays loading state, POSTs JSON to /api/predict, ignores stale responses using request id, stores successful payload; shows a generic network/API error state on failure.
renderDashboard()	Maps response fields to title, date context, service/crowding labels and detailed cards; dispatches chart/table renderers.
renderDemandChart()	Creates/updates the hourly 05-23 demand chart and marks the selected hour; line series may be included for interchange context.
renderInterchangeBreakdown()	Builds line rows and doughnut visualization from backend line allocations.
filterAndDisplayTravelTimes()	Searches and renders destination-time entries returned by API.

HTML safety and consistency

Where dynamic data is inserted with `innerHTML`, station names or API-returned strings should be escaped or rendered with textContent. Station data is largely repository-controlled today, but this becomes a security boundary if user-editable or external data is added.

Frontend/back-end contract checklist

• Keep API field names and types synchronized with renderer expectations.

• Handle HTTP error status before parsing as a successful forecast.

• A full-day profile is not the selected-time count; charts and KPIs should label them distinctly.

• Date preset buttons use the browser date/time context while backend calendar rules use Delhi time; timezone differences are a potential edge case.

• Chart instances should be updated or destroyed before recreation to avoid duplicate canvases/listeners.

11. Error handling, edge cases and common misunderstandings

Case	What code currently does / what to watch
Invalid date or time text	Datetime parser raises; main API returns a client error (400). Browser should show a useful message, not interpret it as zero demand.
Time 00:00-04:59	Selected-time result should be closed/zero via operating-hours gate. Daily operating profile can still display its daytime curve.
23:30 versus 23:31	Boundary is minute-specific; verify callers consistently use hour and minute rather than hour only.
Unknown station	Forecast helper can create an “Unknown” fallback, but graph origin may not exist. Validate station against directory at HTTP boundary for reliable response.
Missing model or metadata	Imports load artifacts at module import, so the WSGI process may fail to boot. Bundle/version artifacts and dependencies together.
Missing large generated CSV	Does not stop normal inference if saved model and small runtime inputs exist; prevents retraining until regenerated/restored.
Station with no lines	Empty line list can cause primary-line selection to fail; validate station metadata and handle empty schedules explicitly.
Historical profile vs current time	The profile visualizes all modeled hours. It does not imply the metro operates outside service hours.
Crowding/capacity label	It is demand versus assumed boarding-clearance capacity, not real occupancy or passenger count.
Travel time	Shortest weighted graph estimate, not a promised journey duration or complete route itinerary.
Model metrics	Synthetic holdout fit, not evidence of accuracy on measured Delhi Metro usage.

Strong code-review observations

• Validate station ids/names at the request boundary, and return consistent 4xx errors for invalid selections.

• Add focused tests around 04:59/05:00 and 23:30/23:31, interchange allocation sums, and graph transfer paths.

• Escape untrusted strings before inserting into HTML.

• Separate configuration assumptions from code and record provenance/version.

• Avoid claiming “real-time,” “actual crowding,” or measured model accuracy unless source data is integrated.

12. Interview practice: walkthroughs and questions

Two-minute technical walkthrough

“MetroInsights is a Flask application that serves a saved scikit-learn model for station-level demand scenarios. Its offline pipeline cleans station metadata, generates reproducible synthetic hourly targets, and trains a HistGradientBoostingRegressor with a chronological holdout. At runtime, Flask validates station/date/time inputs, applies a Delhi calendar and operating-hours context, builds the saved feature schema, predicts demand and interpolates within the hour. At interchanges, generated line shares allocate demand to lines. A deterministic optimizer then translates demand and assumed capacity/headway constraints into service recommendations. A separate topology graph uses Dijkstra to estimate shortest travel times. The frontend fetches JSON and renders KPIs, a demand curve, line breakdown and destination estimates. The key limitation is that training targets and operating assumptions are synthetic or hand-authored, so outputs are planning scenarios rather than live or validated DMRC forecasts.”

Likely questions and answer anchors

Question	Answer anchor
Walk through a /api/predict request.	Browser fetch -> route parse/validation -> calendar + service window -> model/profile -> line allocation + schedule -> Dijkstra -> JSON -> DOM/chart render.
Why separate inference and training?	Training is heavy/offline; serving loads immutable artifacts and only computes a small feature row per request.
What prevents train crowding in this model?	Nothing train-by-train. The optimizer uses demand and assumed boarding clearance; it does not observe onboard occupancy or exits.
Why use Dijkstra?	All edge weights are nonnegative estimated minutes, so Dijkstra provides a shortest weighted travel-time estimate.
Where can leakage enter?	Historical aggregates or category encodings built using future/holdout outcomes. The pipeline attempts to use prior/training-only aggregates; audit that contract.
What would improve validity most?	Observed station/time entries and exits, OD/transfer data, verified service/fleet parameters, and evaluation on a later real-data period.
What is the app’s persistence model?	No database; static files/artifacts are read, request outputs are generated and returned without saved user scenarios.

13. Practical exercises to prove understanding

Use these as small code-reading or implementation exercises. They are suggestions, not claims that the current repository already has tests.

Exercise	What you should inspect or demonstrate
Trace a 09:15 request	Identify route input parsing, feature creation, interpolation between 09:00 and 10:00, service allocation and the JSON fields consumed by the UI.
Prove closure boundaries	Manually inspect behavior at 04:59, 05:00, 23:30 and 23:31 across API selected demand and schedule status. Explain why the chart still contains daytime values.
Explain one feature row	Pick a known station/date/hour; map each feature to saved metadata and verify column order matches the estimator.
Change a synthetic factor	Modify one generator multiplier in a disposable branch; predict which stations/dates/hours change and how the seed affects repeatability.
Trace an interchange allocation	Compare per-line shares to one; follow allocated demand into line schedule and chart breakdown. Explain why it is not a measured transfer split.
Walk a graph edge	Choose adjacent stops on one topology line; calculate distance-based minutes or identify the fallback; explain transfer penalty and Dijkstra relaxation.
Handle invalid station	Design validation before graph/forecast calls; specify status code and a stable JSON error schema.
Add a focused regression test	Test operating-hour boundaries or allocation sum; state the invariant, setup, expected result and why it matters.
Improve deployment reproducibility	Propose Python/dependency pins, artifact version metadata and a clear rebuild procedure while keeping large generated CSV out of serving image.

Suggested source-reading order

1. `run.py` and `app/__init__.py`; 2. `app/routes.py`; 3. `app/operating_hours.py` and `calendar_context.py`; 4. `app/forecasting.py`; 5. `app/scheduling_optimizer.py`; 6. `network_topology.py` and `travel_graph.py`; 7. `dashboard.js`; 8. pipeline 01, 02, then 03.

End of Phase 2. Phase 3 has not been generated.